

Advanced Statistical Inference -- Gaussian Process Regression

In this lab, you will build a Gaussian process regression model from first principles.

By the end of the lab, you should be able to:

- construct and visualize a Gaussian process prior;
- compute the Gaussian process posterior for regression;
- understand how the kernel hyperparameters affect smoothness and uncertainty;
- evaluate hyperparameters with the marginal likelihood;
- optimize hyperparameters with automatic differentiation.

This lab is deliberately structured like the MCMC classification lab: most of the scaffolding is provided, and your job is to complete the missing pieces and interpret the outputs.

How to approach this lab:

- work through the cells in order;
- implement each missing block before moving on;
- after each plot, pause and answer the short conceptual questions;
- compare what you see with the theory from the lecture notes.

Important:

This lab uses JAX throughout. The goal here is not to re-teach JAX syntax. If you need a refresher on `jax.numpy`, `jax.grad`, or `jax.value_and_grad`, revisit the general tutorial:

`tutorials/02_Tutorial_JAX.ipynb`

You should have completed that tutorial before continuing.

Setup

First, import the libraries and configure the plotting style.

```
In [6]: import functools
import warnings

import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
```

```

import numpy as np
from matplotlib import rc

warnings.filterwarnings("ignore")

colab = "google.colab" in str(get_ipython())
rc("figure", **{"dpi": 200})
rc(
    "axes",
    **{"spines.right": False, "spines.top": False, "xmargin": 0.0, "ymargin": 0.0}
)

def plot_data(X, y, ax):
    """Plot 1D regression observations."""
    config = dict(edgecolor="black", linewidth=1, facecolor="tab:blue")
    ax.scatter(X, y, label="Observed data", zorder=10, **config)

def plot_gp(x, mean, cov, palette="Greens", **kwargs):
    """Plot GP mean and uncertainty bands."""
    ax = kwargs.pop("ax", plt.gca())
    cmap = plt.get_cmap(palette)
    ci = [1, 2, 3]
    colors = (ci - np.min(ci)) / (np.max(ci) - np.min(ci) + 3) + 0.1
    x = x.flatten()
    ax.plot(x, mean, color=cmap(0.9), lw=4)
    for i, c in enumerate(ci[:-1]):
        up = mean + c * np.sqrt(np.diag(cov))
        lo = mean - c * np.sqrt(np.diag(cov))
        color = cmap(colors[i])
        ax.fill_between(x, up, lo, color=color, alpha=0.95, **kwargs)
    return ax

def plot_samples(x, samples, palette="Greens", **kwargs):
    """Plot a subset of sampled functions."""
    ax = kwargs.pop("ax", plt.gca())
    N = kwargs.pop("N", 20)
    cmap = plt.get_cmap(palette)
    x = np.asarray(x).reshape(-1)
    samples = np.asarray(samples)
    idx = np.random.randint(0, samples.shape[0], N)
    ax.plot(x, samples[idx].T, color=cmap(0.5), lw=1, alpha=0.75, **kwargs)
    return ax

```

Data

We will work with a one-dimensional regression dataset generated from a smooth latent function with noise. The input region contains a gap on purpose: later, this will make the posterior uncertainty easier to interpret.

Exercise: Run the next cell to generate and visualize the data.

```
In [41]: np.random.seed(3424121)
```

```
def make_dataset(a=-20, b=80, N=64, M=200, gap_ratio=0.4, ampl=1.6, leng=8,
    """Generate a toy 1D regression problem with a gap in the input space."""

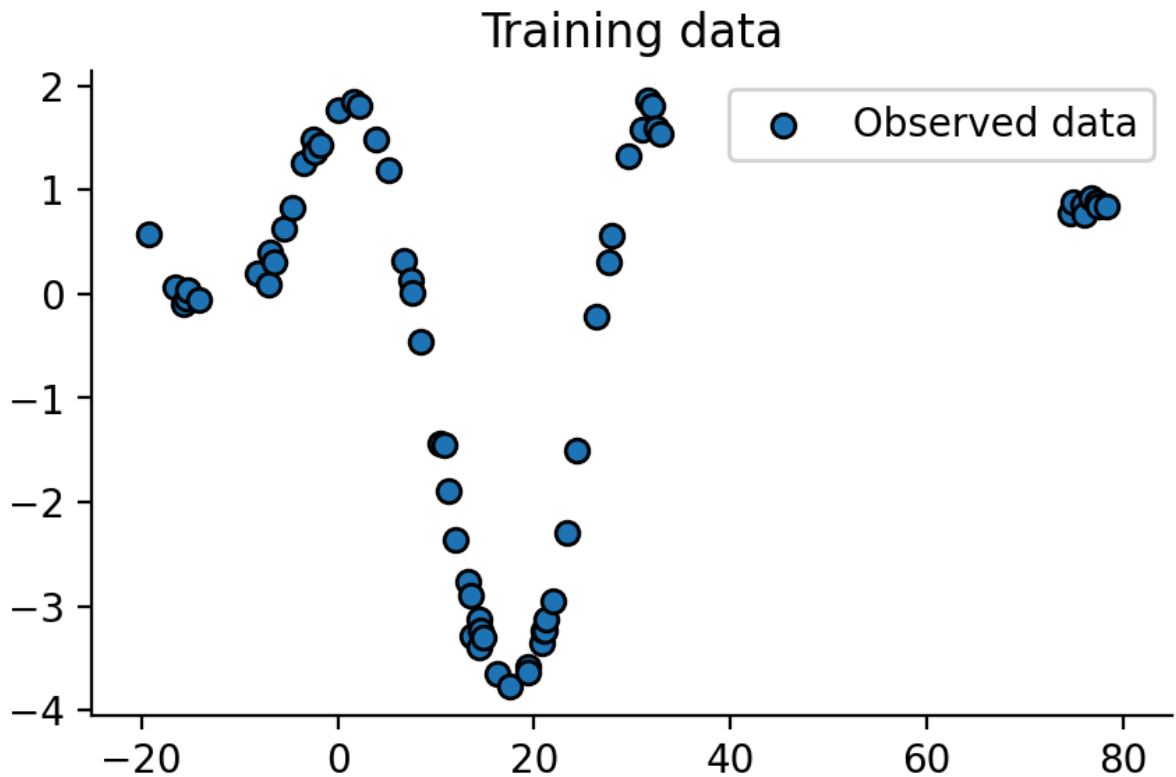
    def make_random_gap(X, gap_ratio=0.2):
        a, b = X.min(), X.max()
        gap_a = a + np.random.rand() * (b - a) * (1 - gap_ratio)
        gap_b = gap_a + (b - a) * gap_ratio
        idx = np.logical_and(gap_a < X, X < gap_b)
        X[idx] = a + np.random.rand(idx.sum()) * (gap_a - a)
        return X

    def sample_random_function(X, ampl=1, leng=1, sn2=0.1):
        n, x = X.shape[0], X / leng
        sum_xx = np.sum(x * x, 1).reshape(-1, 1).repeat(n, 1)
        D = sum_xx + sum_xx.transpose() - 2 * np.matmul(x, x.transpose())
        C = ampl**2 * np.exp(-0.5 * D) + np.eye(n) * sn2
        return np.random.multivariate_normal(np.zeros(n), C)

    X = np.random.rand(N, 1) * (b - a) + a
    X = make_random_gap(X, gap_ratio=gap_ratio)
    ind = np.argsort(X[:, 0])
    y = sample_random_function(X, ampl=ampl, leng=leng, sn2=sn2)
    Xt = np.linspace(a - 5, b + 5, M).reshape(-1, 1)
    return X[ind], y[ind], Xt

X, y, Xt = make_dataset()

fig, ax = plt.subplots(figsize=[5, 3])
plot_data(X, y, ax)
ax.set_xlim(Xt.min(), Xt.max())
ax.set_title("Training data")
ax.legend()
plt.show()
```



Questions:

- Where is the input gap?
- In which region would you expect the posterior uncertainty to be largest?
- Do you expect a very small or a very large lengthscale to fit this dataset better?

The input gap is between 35 and 75 and the posterior uncertainty is expected to be largest in this region. A very small lengthscale would not fit the data well, as it would lead to a very wiggly function that overfits the noise. A very large lengthscale would also not fit the data well, as it would lead to a very smooth function that underfits the data. Therefore, we would expect an intermediate lengthscale to fit this dataset better.

1. Gaussian Process Prior

A Gaussian process defines a distribution over functions. For any finite set of inputs, the corresponding function values follow a multivariate Gaussian:

$$p(\mathbf{f}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

In this lab, we use a zero-mean prior and an RBF kernel:

$$\kappa(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right).$$

The kernel has two main hyperparameters:

- `lengthscale` l : controls how quickly correlations decay with distance;
- `variance` σ_f^2 : controls the vertical scale of the function.

Exercise: Complete the `cdist()` and `rbf_kernel()` functions.

`cdist(A, B)` should return the matrix of pairwise squared Euclidean distances between rows of `A` and rows of `B`.

```
In [42]: def cdist(A, B):
    """Compute the matrix of pairwise squared Euclidean distances."""
    M = A.shape[0]
    N = B.shape[0]
    # @@ COMPLETE @@
    A_dots = jnp.sum(A * A, axis=1).reshape(-1, 1).repeat(N, axis=1)
    B_dots = jnp.sum(B * B, axis=1).reshape(1, -1).repeat(M, axis=0)
    D_squared = A_dots + B_dots - 2 * jnp.dot(A, B.T)
    return D_squared

def rbf_kernel(params, X1, X2):
    """Compute the RBF covariance matrix between two sets of inputs."""
    lengthscale = params["lengthscale"]
    variance = params["variance"]
    # @@ COMPLETE @@
    out = variance * jnp.exp(-0.5 * cdist(X1 / lengthscale, X2 / lengthscale))
    return out
```

Exercise: Compute the covariance matrix $\kappa(\mathbf{X}, \mathbf{X})$ using:

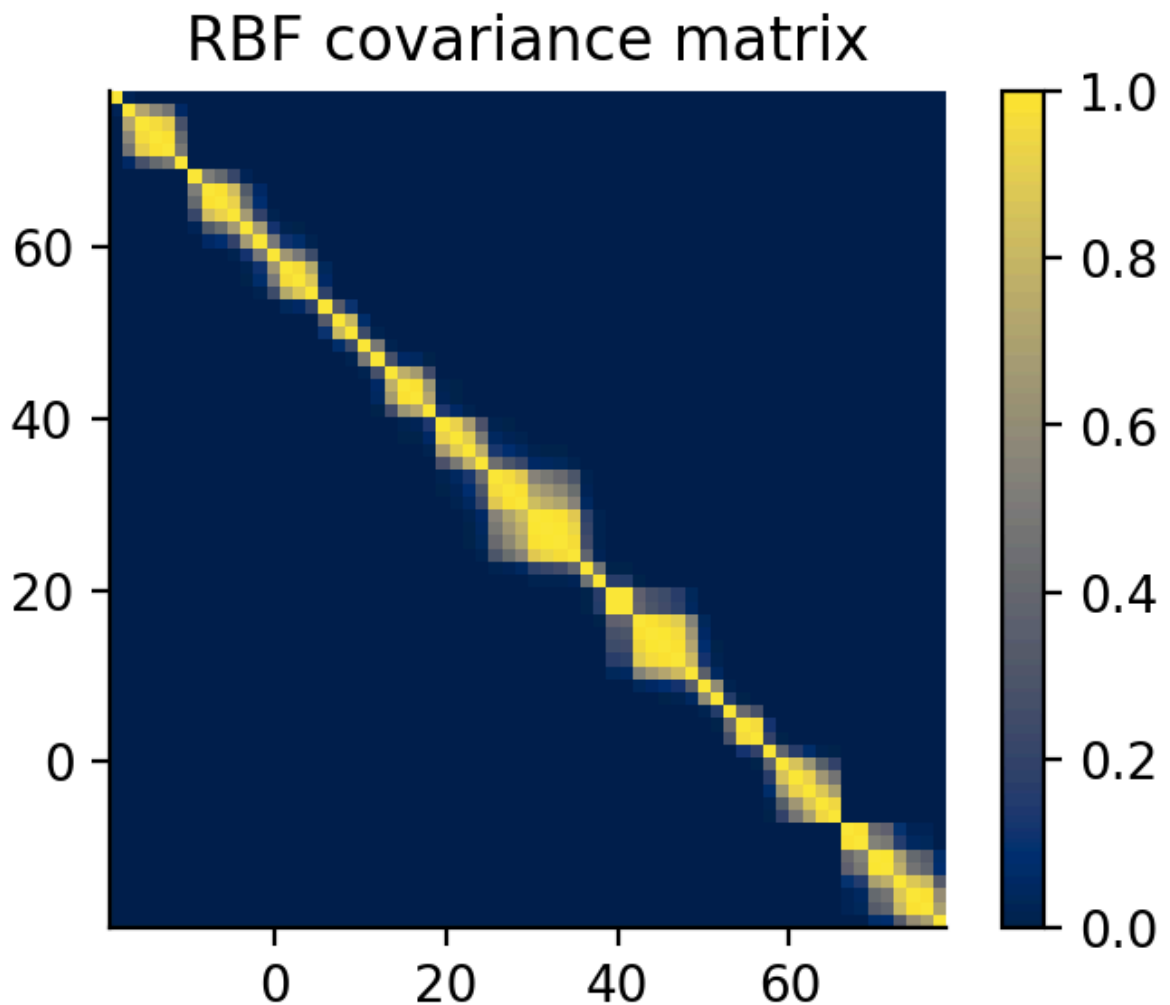
- `lengthscale = 1.0`
- `variance = 1.0`

Then plot it with `plt.imshow()`.

```
In [43]: params = {"lengthscale": 1.0, "variance": 1.0}

# @@ COMPLETE @@
K = rbf_kernel(params, X, X)

fig, ax = plt.subplots(figsize=[4, 3])
image = ax.imshow(K, extent=[X.min(), X.max(), X.min(), X.max()], cmap="civid")
fig.colorbar(image)
ax.set_title("RBF covariance matrix")
plt.show()
```



Questions:

- If your implementation is correct, what should the diagonal entries be equal to?
- What happens to the off-diagonal values when points are far apart?
- If you increase the lengthscale, should the covariance matrix become more local or more global?

the diagonal entries should be equal to the variance, which is 1.0 in this case. The off-diagonal values will become smaller as the points are farther apart, due to the exponential decay in the RBF kernel. If you increase the lengthscale, the covariance matrix will become more global, meaning that points will be more correlated even if they are farther apart.

To sample from the GP prior over function values at test inputs $\mathbf{X}_t$, we use:

$$\mathbf{f}_t \sim \mathcal{N}(\mathbf{0}, \kappa(\mathbf{X}_t, \mathbf{X}_t)).$$

Exercise: Complete the function below to sample functions from the GP prior.

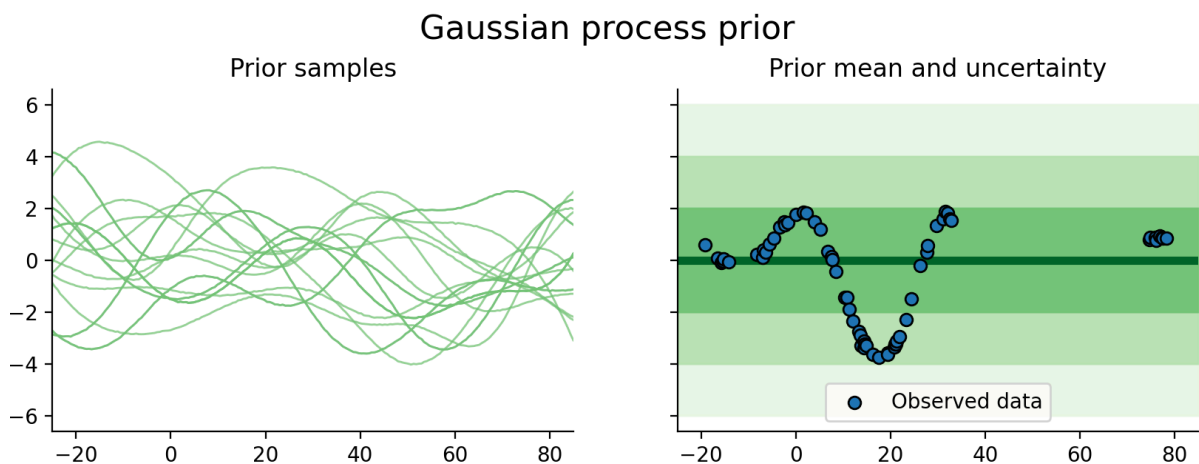
```
In [71]: def sample_f_prior(params, kernel_fn, Xt, N=20):
        """Sample N functions from a zero-mean GP prior evaluated at Xt."""
        # @@ COMPLETE @@
        mean = jnp.zeros(Xt.shape[0])
        cov = kernel_fn(params, Xt, Xt)

        # numerical stability
        cov = 0.5 * (cov + cov.T)
        cov = cov + 1e-4 * jnp.eye(Xt.shape[0])

        samples = jax.random.multivariate_normal(jax.random.PRNGKey(0), mean, cov)
        return samples, mean, cov

params = {"lengthscale": 15, "variance": 4.0}
samples, mean, cov = sample_f_prior(params, rbf_kernel, Xt)

fig, (ax0, ax1) = plt.subplots(1, 2, figsize=[10, 3], sharey=True)
plot_samples(Xt, samples, ax=ax0)
plot_gp(Xt, mean, cov, ax=ax1)
plot_data(X, y, ax1)
ax0.set_title("Prior samples")
ax1.set_title("Prior mean and uncertainty")
ax1.legend()
fig.suptitle("Gaussian process prior", y=1.05, fontsize=16)
plt.show()
```



Questions:

- Are the prior samples smooth or jagged?
- What happens when you decrease the lengthscale?
- What happens to the vertical spread when you increase the variance?

The prior samples are still smooth, but they become more wiggly when the lengthscale decreases. When the lengthscale is smaller, correlations decay faster with distance in (x). That allows more local variation, so functions fluctuate more, even though they remain smooth/continuous. When the variance increases, the marginal variance of function values increases. This makes samples spread out more in the (y)-direction and

widens the uncertainty bands. In short: smaller lengthscale gives more local wiggles; larger variance gives larger vertical

For very small lengthscales and high variance, I also observe numerical instability in the GP prior sampling (the covariance matrix becomes poorly conditioned), which can lead to unstable samples or plotting

Exercise: Try at least three different hyperparameter settings and compare the prior samples. A useful set to try is:

- `(lengthscale=3.0, variance=1.0)`
- `(lengthscale=15.0, variance=1.0)`
- `(lengthscale=15.0, variance=4.0)`

2. Posterior Inference

Once we observe data, the prior is updated into a posterior distribution over functions.

For Gaussian process regression, the posterior at test inputs `Xt` is again Gaussian. The main computation involves kernel matrices and solving a linear system. To do this stably, we use a Cholesky decomposition instead of a direct matrix inverse.

Exercise: Complete the function `predict_f_posterior()`.

Use the following notation:

- `K = k(X, X) + \sigma_n^2 I`
- `Kx = k(X, Xt)`
- `Kxx = k(Xt, Xt)`

Then compute the posterior mean and covariance using triangular solves.

```
In [77]: def predict_f_posterior(params, kernel_fn, X, y, Xt, sn2=0.01):
    """Compute the GP posterior mean and covariance at test inputs Xt."""
    # @@ COMPLETE @@
    K = kernel_fn(params, X, X)
    K = 0.5 * (K + K.T) + (sn2 + 1e-6) * jnp.eye(X.shape[0])
    Kx = kernel_fn(params, X, Xt)
    Kxx = kernel_fn(params, Xt, Xt)
    Kxx = 0.5 * (Kxx + Kxx.T)
    L = jnp.linalg.cholesky(K)
    z = jax.scipy.linalg.solve_triangular(L, y, lower=True)
    a = jax.scipy.linalg.solve_triangular(L.T, z, lower=False)
    fmean = Kx.T @ a
    v = jax.scipy.linalg.solve_triangular(L, Kx, lower=True)
    fcov = Kxx - v.T @ v
    fcov = 0.5 * (fcov + fcov.T) + 1e-6 * jnp.eye(Xt.shape[0])
    return fmean, fcov
```


We first condition on only a few observations. This makes it easier to see how the posterior differs from the prior.

Exercise: Select a small training subset by sampling 5 indices at random.

```
In [73]: np.random.seed(14)
idx = np.random.permutation(len(X))[:5]

# @@ COMPLETE @@
Xtr = X[idx]
ytr = y[idx]

print(f"Training points: {Xtr.flatten()}")
print(f"Training values: {ytr.flatten()}")
```

```
Training points: [ -8.18128681  13.25811133  77.25791517  14.56215491 -15.74
802938]
Training values: [ 0.20594225 -2.76920918  0.8820364  -3.23330333 -0.0915185
]
```

Exercise: Compute the posterior mean and covariance at all test inputs `Xt`.

```
In [78]: # @@ COMPLETE @@
f_mean, f_cov = predict_f_posterior(params, rbf_kernel, Xtr, ytr, Xt)

print(f"Posterior mean shape: {f_mean.shape}")
print(f"Posterior covariance shape: {f_cov.shape}")
```

```
Posterior mean shape: (200,)
Posterior covariance shape: (200, 200)
```

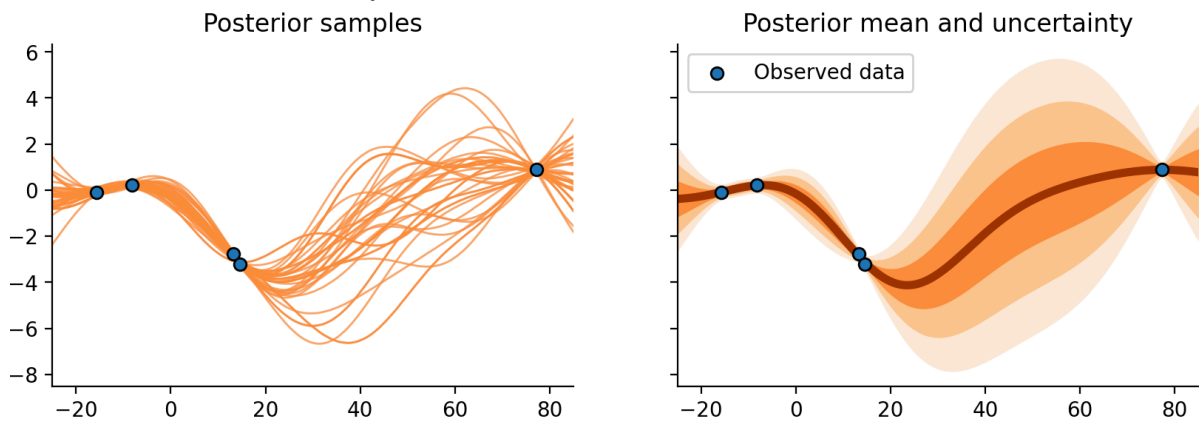
Exercise: Draw posterior samples and visualize them together with the posterior uncertainty.

```
In [82]: np.random.seed(0)
n_samples = 50

# @@ COMPLETE @@
f_cov_stable = 0.5 * (f_cov + f_cov.T)
eigvals, eigvecs = jnp.linalg.eigh(f_cov_stable)
eigvals = jnp.clip(eigvals, a_min=1e-8)
z = jax.random.normal(jax.random.PRNGKey(0), shape=(n_samples, f_mean.shape[0]))
samples = f_mean[None, :] + z @ (eigvecs * jnp.sqrt(eigvals)).T

fig, (ax0, ax1) = plt.subplots(1, 2, figsize=[10, 3], sharey=True)
plot_samples(Xt, samples, palette="Oranges", ax=ax0, N=40)
plot_data(Xtr, ytr, ax=ax0)
plot_gp(Xt, f_mean, f_cov_stable, ax=ax1, palette="Oranges")
plot_data(Xtr, ytr, ax=ax1)
ax0.set_title("Posterior samples")
ax1.set_title("Posterior mean and uncertainty")
ax1.legend()
fig.suptitle(f"GP posterior with N={len(Xtr)} observations", y=1.05, fontsize=12)
plt.show()
```

GP posterior with N=5 observations



Questions:

- Does the posterior uncertainty shrink near observed points?
- In the input gap, is the uncertainty larger or smaller than near the observations?
- Do the sampled functions interpolate the observed points exactly, or only approximately? Why?

Yes the posterior uncertainty shrinks near observed points because the model is more confident about the function values in those regions. In the input gap, the uncertainty is larger than near the observations because there are no data points to constrain the function values, leading to greater uncertainty. The sampled functions interpolate the observed points exactly because we are using a Gaussian process regression model, which assumes that the function values at the observed points are known with certainty (i.e., they are not noisy). Therefore, the posterior mean will pass through the observed points, and the samples will also interpolate them exactly.

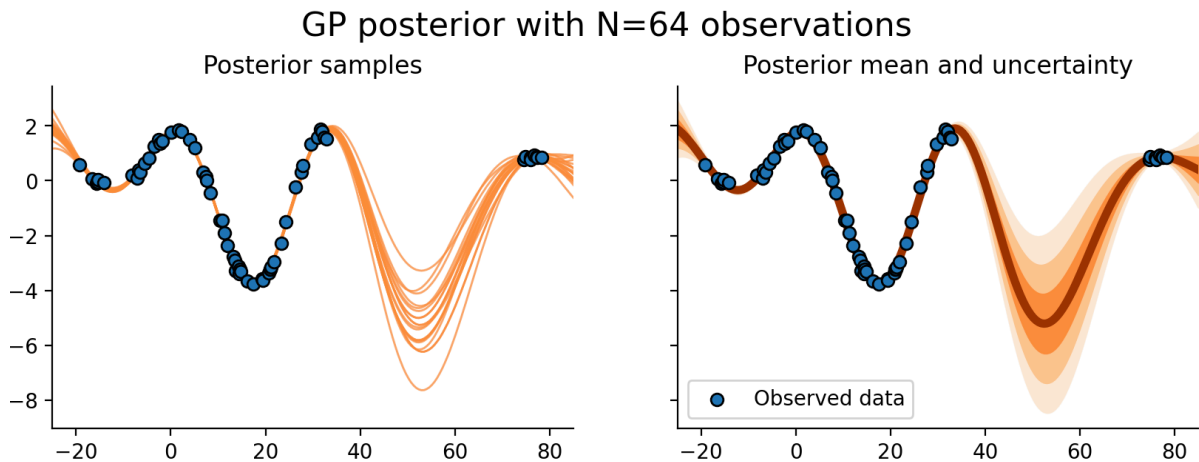
Exercise: Now condition on the full dataset and compare the posterior to the previous plot.

```
In [85]: np.random.seed(0)

# @@ COMPLETE @@
Xtr, ytr = X, y
f_mean, f_cov = predict_f_posterior(params, rbf_kernel, Xtr, ytr, Xt)
n_samples = 50
f_cov_stable = 0.5 * (f_cov + f_cov.T)
eigvals, eigvecs = jnp.linalg.eigh(f_cov_stable)
eigvals = jnp.clip(eigvals, a_min=1e-8)
z = jax.random.normal(jax.random.PRNGKey(0), shape=(n_samples, f_mean.shape[0]))
samples = f_mean[None, :] + z @ (eigvecs * jnp.sqrt(eigvals)).T

fig, (ax0, ax1) = plt.subplots(1, 2, figsize=[10, 3], sharey=True)
plot_samples(Xt, samples, palette="Oranges", ax=ax0)
plot_data(X, y, ax=ax0)
plot_gp(Xt, f_mean, f_cov_stable, ax=ax1, palette="Oranges")
plot_data(X, y, ax=ax1)
```

```
ax0.set_title("Posterior samples")
ax1.set_title("Posterior mean and uncertainty")
ax1.legend()
fig.suptitle(f"GP posterior with N={len(Xtr)} observations", y=1.05, fontsize=12)
plt.show()
```



Questions:

- Compared with the 5-point posterior, where did the uncertainty decrease the most?
- Does the gap still look more uncertain than densely observed regions?
- If your code is correct, does the posterior mean follow the global trend of the data?

the uncertainty decreased the most in the regions where we added more observations, especially in the input gap. The gap still looks more uncertain than densely observed regions, but the uncertainty is reduced compared to the 5-point posterior. If the code is correct, the posterior mean should follow the global trend of the data, capturing the underlying function that generated the observations.

Exercise: Try 3 or 4 kernel hyperparameter combinations and compare the posterior each time.

Focus on these questions:

- What kind of posterior mean do you get with a very short lengthscale?
- What kind of posterior mean do you get with a very long lengthscale?
- Which setting seems most plausible for this dataset?

3. Model Evaluation with the Marginal Likelihood

A Gaussian process does not only make predictions. It also assigns a probability to the observed data under a given choice of hyperparameters.

This is the marginal likelihood:

$$\log p(\mathbf{y} \mid \mathbf{X}) = \log \mathcal{N}(\mathbf{y} \mid \mathbf{0}, \kappa(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}).$$

It provides a principled way to compare hyperparameter settings.

Exercise: Complete the function below to compute the log marginal likelihood using a Cholesky factorization.

```
In [86]: @functools.partial(jax.jit, static_argnums=(1,))
def marginal_likelihood(params, kernel_fn, X, y, sn2=0.01):
    """Compute the log marginal likelihood of the GP model."""
    # @@ COMPLETE @@
    K = kernel_fn(params, X, X) + sn2 * jnp.eye(X.shape[0])
    L = jnp.linalg.cholesky(K)
    V = jax.scipy.linalg.solve_triangular(L, y, lower=True)
    model_fit_term = -0.5 * jnp.dot(V, V)
    log_det_term = -jnp.sum(jnp.log(jnp.diag(L)))
    ll = model_fit_term + log_det_term - 0.5 * X.shape[0] * jnp.log(2 * jnp.
    return ll
```

Exercise: Evaluate the marginal likelihood for one hyperparameter setting.

```
In [87]: params = {"lengthscale": 15.0, "variance": 1.0}
print(marginal_likelihood(params, rbf_kernel, X, y))
```

-237.15689

Exercise: Run a grid search over `lengthscale` and `variance`, then visualize the marginal likelihood surface.

```
In [88]: lengthscales = np.logspace(-0.5, 1.5, 100)
variances = np.logspace(-0.5, 1.5, 100)
marginal_ll = np.empty((len(lengthscales), len(variances)))

for i, lengthscale in enumerate(lengthscales):
    for j, variance in enumerate(variances):
        _params = {"lengthscale": lengthscale, "variance": variance}
        # @@ COMPLETE @@
        marginal_ll[i, j] = marginal_likelihood(_params, rbf_kernel, X, y)

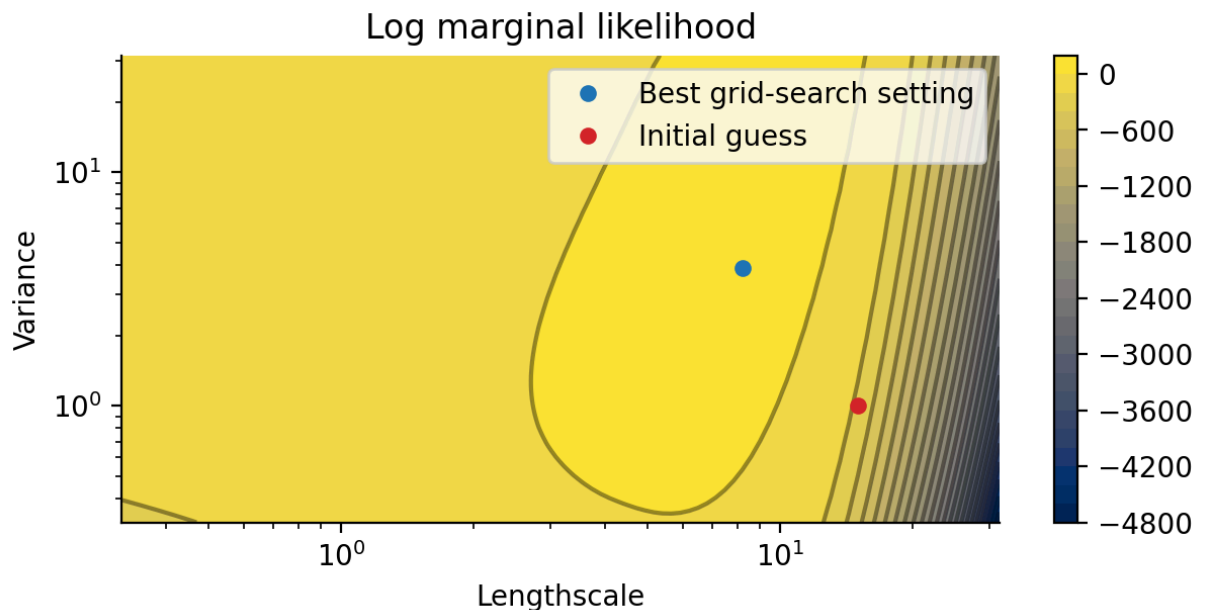
best = np.unravel_index(marginal_ll.argmax(), marginal_ll.shape)

fig, ax = plt.subplots(figsize=(7, 3))
cset = ax.contourf(lengthscales, variances, marginal_ll.T, cmap="cividis", 1
fig.colorbar(cset)
ax.contour(
    lengthscales,
    variances,
    marginal_ll.T,
    colors="k",
    linestyle="-",
    alpha=0.4,
    levels=30,
)
```

```

ax.plot(
    lengthscales[best[0]],
    variances[best[1]],
    "o",
    color="tab:blue",
    ms=5,
    label="Best grid-search setting",
)
ax.plot(
    params["lengthscale"],
    params["variance"],
    "o",
    color="tab:red",
    ms=5,
    label="Initial guess",
)
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("Lengthscale")
ax.set_ylabel("Variance")
ax.set_title("Log marginal likelihood")
ax.legend()
plt.show()

```



Questions:

- Is the best region associated with very short, intermediate, or very long lengthscales?
- Does the best variance look tiny, huge, or moderate?
- Do the best hyperparameters match your visual intuition from the posterior plots?

the best region is associated with intermediate lengthscales, as very short lengthscales would lead to overfitting and very long lengthscales would lead to underfitting. The best variance looks moderate, as a tiny variance would not capture the variability in the data,

while a huge variance would lead to a very wide posterior that does not fit the data well. The best hyperparameters should match the visual intuition from the posterior plots, where we expect a smooth function that captures the underlying trend without being too wiggly or too flat.

4. Hyperparameter Optimization with Gradients

Instead of evaluating a large grid, we can optimize the marginal likelihood directly.

You already saw in the JAX tutorial how `jax.grad` and `jax.value_and_grad` work. Here we apply them to GP hyperparameters.

Exercise: Create a function that returns both the marginal likelihood and its gradient with respect to the parameter dictionary.

```
In [89]: # @@ COMPLETE @@
grad_marginal_likelihood = jax.grad(marginal_likelihood, argnums=0)

grad_marginal_likelihood = jax.jit(grad_marginal_likelihood, static_argnums=
```

We now implement one gradient-ascent update step.

```
In [90]: @jax.jit
def gradient_update(params, gradients, learning_rate):
    """Update GP hyperparameters with one gradient-ascent step."""
    updated_params = jax.tree.map(
        lambda p, g: p + learning_rate * g,
        params,
        gradients,
    )
    return updated_params
```

Exercise: Complete the optimization loop and keep track of the marginal likelihood values.

```
In [91]: params = {"lengthscale": 1.0, "variance": 1.0}
learning_rate = 0.001
marginals = []

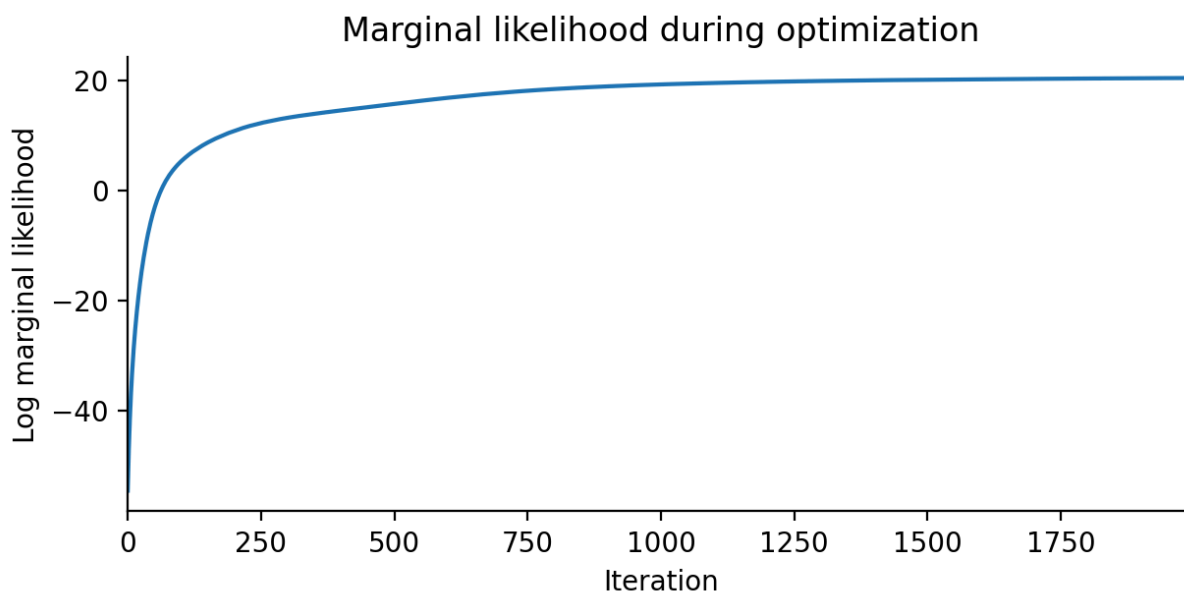
for _ in range(2000):
    # @@ COMPLETE @@
    value, gradients = jax.value_and_grad(marginal_likelihood, argnums=0)(pa
    params = gradient_update(params, gradients, learning_rate)
    marginals.append(value)

params
```

```
Out[91]: {'lengthscale': Array(7.217506, dtype=float32, weak_type=True),
          'variance': Array(2.287624, dtype=float32, weak_type=True)}
```

Exercise: Plot the marginal likelihood during optimization.

```
In [92]: fig, ax = plt.subplots(figsize=[7, 3])
ax.plot(marginals)
ax.set_title("Marginal likelihood during optimization")
ax.set_xlabel("Iteration")
ax.set_ylabel("Log marginal likelihood")
plt.show()
```



Questions:

- Does the objective increase over time?
- Does it improve quickly at the beginning and then plateau?
- If the curve is unstable, what might you change first: the learning rate or the number of iterations?

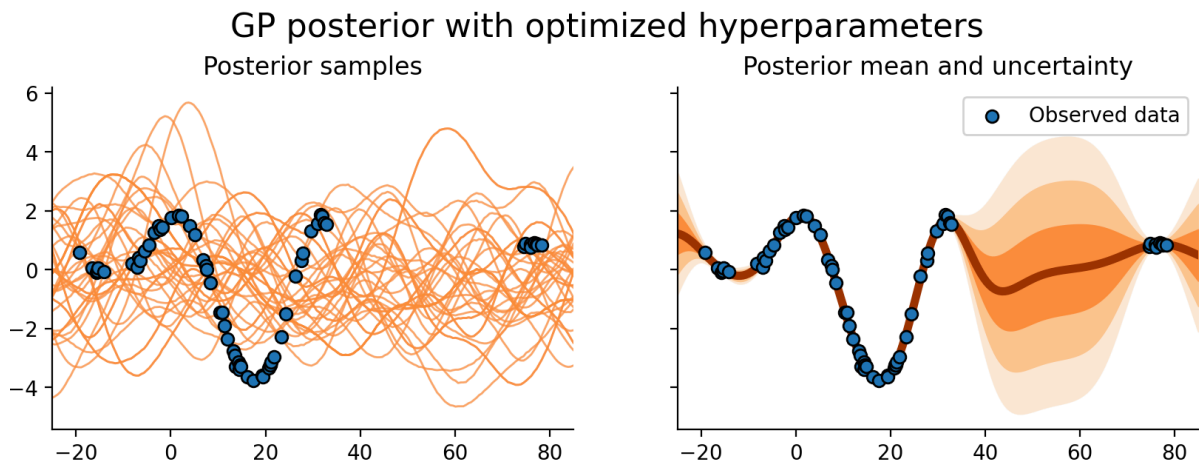
The objective increase over time, showing that the optimization is working. It improves quickly at the beginning and then plateaus as it approaches a local maximum. If the curve is unstable, I would first try reducing the learning rate to ensure more stable updates. If that doesn't help, I might also consider increasing the number of iterations to allow for more gradual convergence.

Exercise: Using the optimized hyperparameters, compute and visualize the posterior again.

```
In [93]: np.random.seed(0)

# @@ COMPLETE @@
f_mean, f_cov = predict_f_posterior(params, rbf_kernel, X, y, Xt)
samples = sample_f_prior(params, rbf_kernel, Xt, N=100)[0]
```

```
fig, (ax0, ax1) = plt.subplots(1, 2, figsize=[10, 3], sharey=True)
plot_samples(Xt, samples, palette="Oranges", ax=ax0, N=40)
plot_data(X, y, ax=ax0)
plot_gp(Xt, f_mean, f_cov, ax=ax1, palette="Oranges")
plot_data(X, y, ax=ax1)
ax0.set_title("Posterior samples")
ax1.set_title("Posterior mean and uncertainty")
ax1.legend()
fig.suptitle("GP posterior with optimized hyperparameters", y=1.05, fontsize=14)
plt.show()
```



Final questions:

- Compared with your earlier posterior plots, does this posterior look more plausible?
- Where is the model still uncertain, and why?
- Are the optimized hyperparameters broadly consistent with the best region of the grid search?

The posterior looks more plausible compared to the earlier plots, as it should better capture the underlying function that generated the data. The model is still uncertain in regions where there are fewer observations, such as the input gap, because there is less information to constrain the function values in those areas. The optimized hyperparameters should be broadly consistent with the best region of the grid search, as both methods aim to maximize the marginal likelihood. However, the optimization may find a slightly different set of hyperparameters due to the continuous nature of the optimization process compared to the discrete grid search.

Optional extension

If you want to go one step further, try one of the following:

- implement a Mat'ern kernel and compare it to the RBF kernel;
- optimize in log-parameter space to guarantee positivity of the hyperparameters;
- vary the observation noise `sn2` and study its effect on the posterior.